



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF SOFTWARE TECHNOLOGY AND METHODOLOGY

Web-Based Automated Comic Generation From Text (Single Character Focus)

Supervisor:

Morse Gregory Reynolds

Assistant Lecturer

Author:

Salah Uddin

Computer Science BSc

Budapest, 2025

Contents

1	Introduction	2
2	User documentation	3
3	Developer documentation	4
3.1	Design	4
3.1.1	Design Overview	4
3.1.2	System Architecture	5
3.1.3	Client-Side Architecture (Front End)	6
3.1.4	Server-Side Architecture (Back End)	10
3.1.5	Data Management & Storage (Database)	11
3.1.6	System Interaction	13
3.2	Implementation	15
4	Conclusion	16
	Acknowledgements	17
A	Simulation results	18
	Bibliography	19
	List of Figures	19
	List of Tables	20
	List of Algorithms	21
	List of Codes	22

Chapter 1

Introduction

Chapter 2

User documentation

Chapter 3

Developer documentation

3.1 Design

3.1.1 Design Overview

The design of our system follows the **Microservices Architecture with a Layered Approach**, ensuring a structured and efficient approach to managing user interactions, data processing, and AI-driven image generation. The primary goal is to provide a user-friendly and intuitive interface for generating high-quality comic-style images while maintaining smooth navigation and accessibility.

This system is designed to cater to three main types of users:

- **Guests (Unregistered Users):** Can access limited features such as previewing AI-generated comics.
- **Registered Users:** Have access to additional features such as saving, downloading, and retrieving their generated comics.
- **Admin:** Manages registered users and supervises system functionality.

The system is divided into three key components:

- **Front-End (Client-Side):** Provides a seamless and interactive user experience.
- **Back-End (Server-Side):** Handles AI model processing, user authentication, and system logic.

- **Database Management:** Ensures secure storage and retrieval of user data and generated comics.

This structured design ensures **scalability, efficiency, and ease of use**, creating a smooth experience for users while maintaining strong system integrity and performance.

3.1.2 System Architecture

The best architecture for this web-based system where users can provide text descriptions and generate comic-style images while focusing on a single character would be the one that allows modularity, scalability, and the ability to integrate AI models efficiently and that is **Microservices architecture**.

Microservices architecture decomposes the application into smaller, independent services that communicate through well-defined APIs. Thus, choosing it offered me several advantages:

- **Scalability:** As my project grows, microservices allow me to scale different components independently.
- **Flexibility:** I can mix and match technologies according to the needs of each service.
- **Maintainability:** Each service is loosely coupled and can be developed and deployed independently, making the system easier to maintain and update.
- **Failure Isolation:** If one component fails, others can continue to operate, improving overall system reliability.
- **Security:** It will ensure the security of my system by limiting the impact of breaches, and enabling independent security controls for each component.

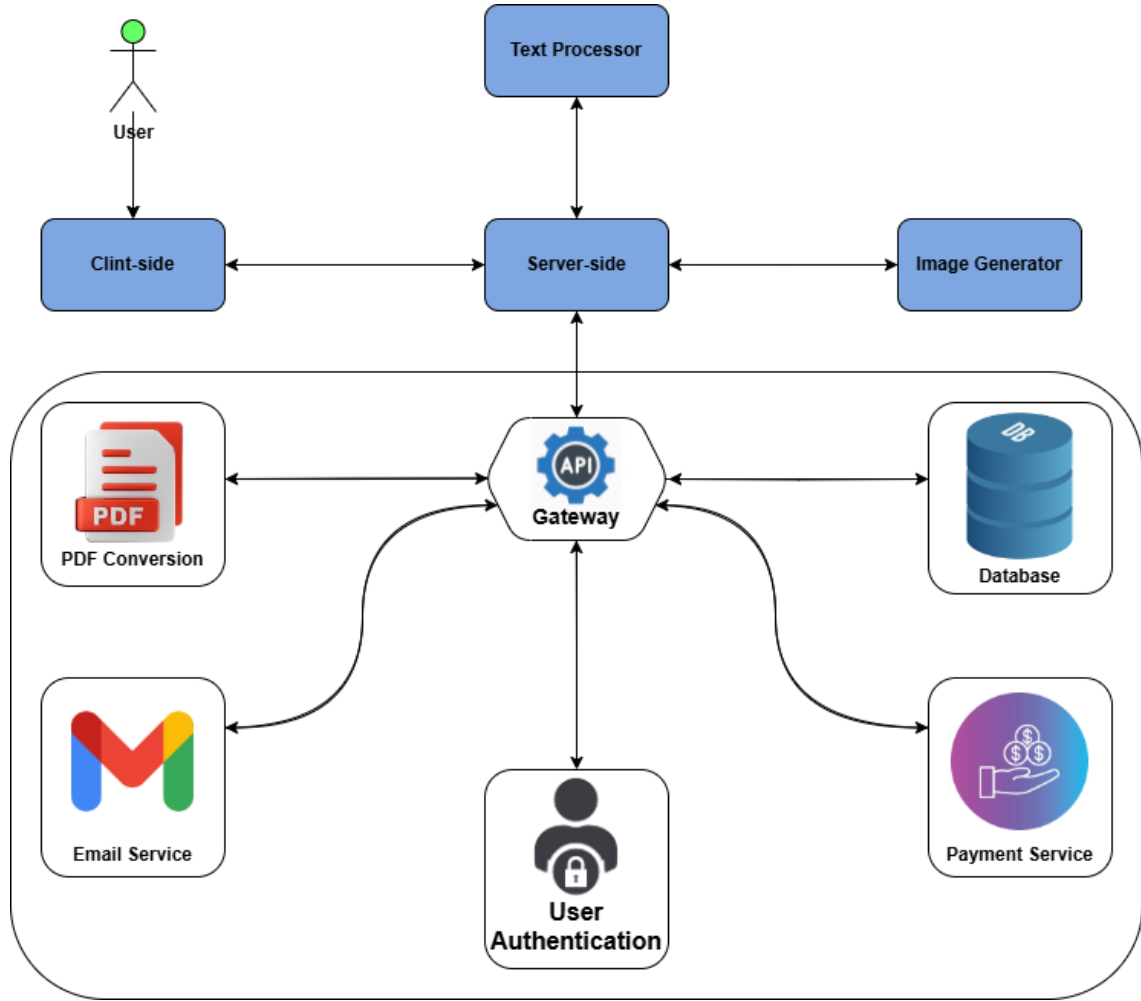


Figure 3.1: System Architecture

Using a **microservices architecture** with an **API gateway** and **service registry** helps solve some challenges such as, **creating a centralized, secure, and scalable system**. By relying on a third-party **authentication service** and **payment service**, the team could focus on the main business features while still maintaining strong security.

3.1.3 Client-Side Architecture (Front End)

User Interface Layout

The system is designed to be super user-friendly, ensuring a smooth and intuitive experience for all users. The layout focuses on simplicity, accessibility, and efficiency, making it easy for users to navigate and interact with the platform. Regardless of the user's role, the interface maintains a consistent design while adapting to different functionalities.

The design is divided into three main layouts, each serving a different purpose:

- **Guest Users** – Can generate a single image at a time such as generate a comic character but cannot save, download, or access previous creations directly.

A Guest User can access the following:

1. Text Input – Can enter a short text prompt to generate a comic-style image.
2. Generate a Single Image – Can create only one image (no multi-panel comics).
3. Preview the Image – Can view the generated image on the website.
4. Basic UI Navigation – Can explore the website interface but with restricted access to user-specific features.
5. Registration Prompt – Encouraged to sign up for full access to features like saving, downloading, and multi-panel comics.

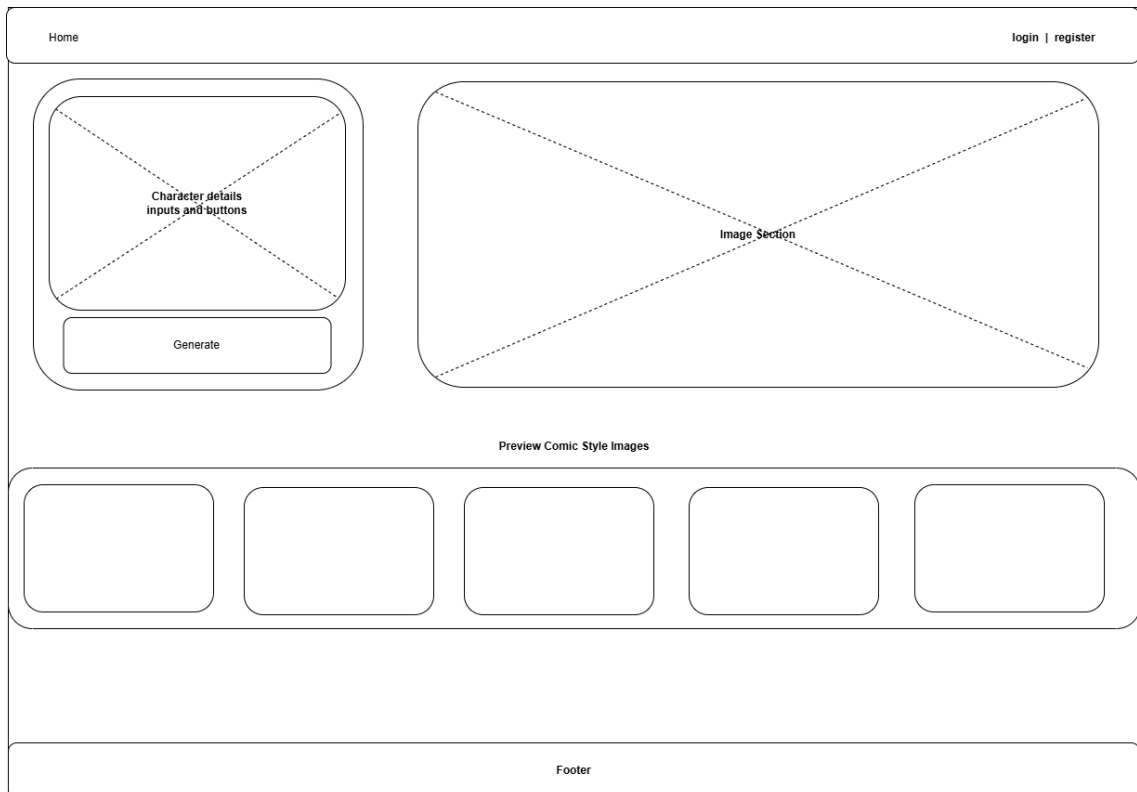


Figure 3.2: Guest Users Front End Diagram

- **Registered Users** – Have full access to comic generation, history management, downloads, and email delivery, allowing them to save and revisit their comics.

A Registered User has access to the following sections and features:

1. Home – Generate multiple images and full comics, with player description, image slider, and download options and also the all features which Guest Users have but more flexible.
2. Profile – View and update user details.
3. My Comics – Access, re-download, or delete previously generated comics.
4. Buy Credits – Purchase credits for generating multi-images.
5. Settings - Manage account preferences such as password, notifications, and privacy settings.
6. Logout - Securely logs the user out and redirects them to the guest interface.

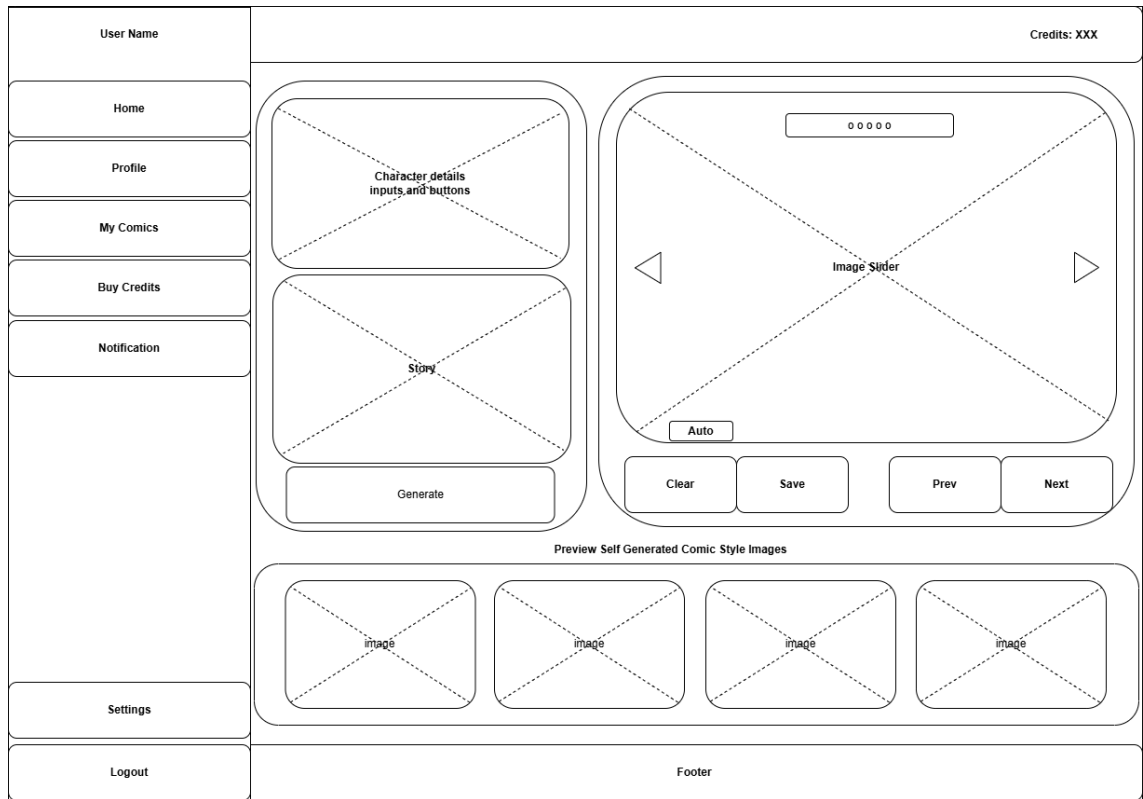


Figure 3.3: Registered Users Front End Diagram

- **Admin** – Oversees the system, manages users, monitors System performance, and handles to ensure smooth operations.
 1. Dashboard – Overview of system activity, user stats, etc.
 2. User Management – View, edit, delete user accounts, reset passwords, and manage permissions.
 3. Comic Moderation – Review, approve, or remove generated comics if needed.
 4. Payment & Credit Management – Monitor transactions, manage refunds, adjust credit pricing, and handle subscriptions.
 5. Email Notifications – Send system updates, promotional emails, or alerts to users.
 6. Backup & Data Management – Manage database backups and ensure data integrity.
 7. Monitoring Mode – Switch between User Mode (to see the platform as users do) and Admin Mode (to detect and fix issues before users experience them).

3.1.4 Server-Side Architecture (Back End)

Use Case Diagram



Figure 3.4: Use Case Diagram

API Design

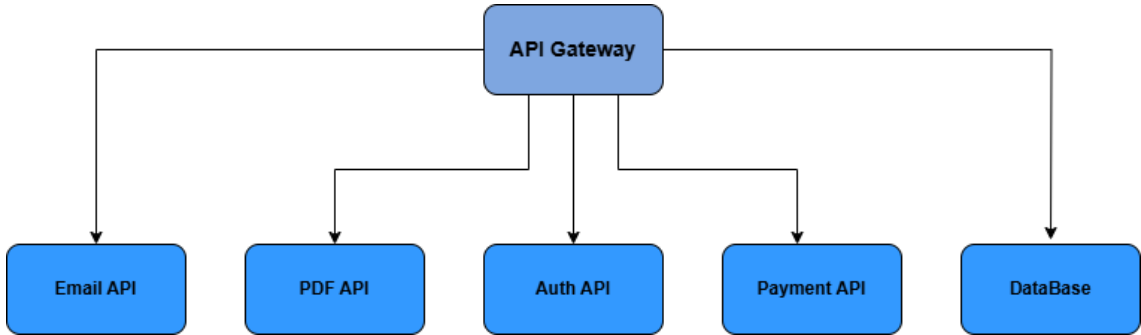


Figure 3.5: API Design Of the System

3.1.5 Data Management & Storage (Database)

Our system has a relational database designed to support user management, comic generation, and payment transactions. ORM (Object-Relational Mapping) is used to simplify database interactions, making data access and manipulation easier.

The System utilizes a relational database schema with the following tables:

1. **Users Table:** This table holds all the information about users, whether they are regular users or admins. It stores their name, email, password, role (user or admin), credits, and account creation date. The system checks if the user is an admin to give special permissions.
2. **Comics Table:** This table stores the comics that users create. It tracks the user who created each comic, the comic's title, the text used to generate the comic, the paths to the generated comic images, the number of pages, and the date it was created.
3. **Admin Table:** Admins also exist in the Users table, but they have an extended profile in a separate table to store their specific permissions. This table tracks what actions the admin is allowed to perform in the system, such as managing users, moderating comics, or handling payments.
4. **Payments Table:** This table records transactions when users purchase credits for generating comics. It includes the amount paid, the credits added to the

user's account, the status of the transaction and the payment date. Admins can monitor and manage payment activities.

Entity Relationship Diagram (ERD)

The 1 - 1 represents the one-to-one relationship and 1 - n shows the one-to-many relationships.

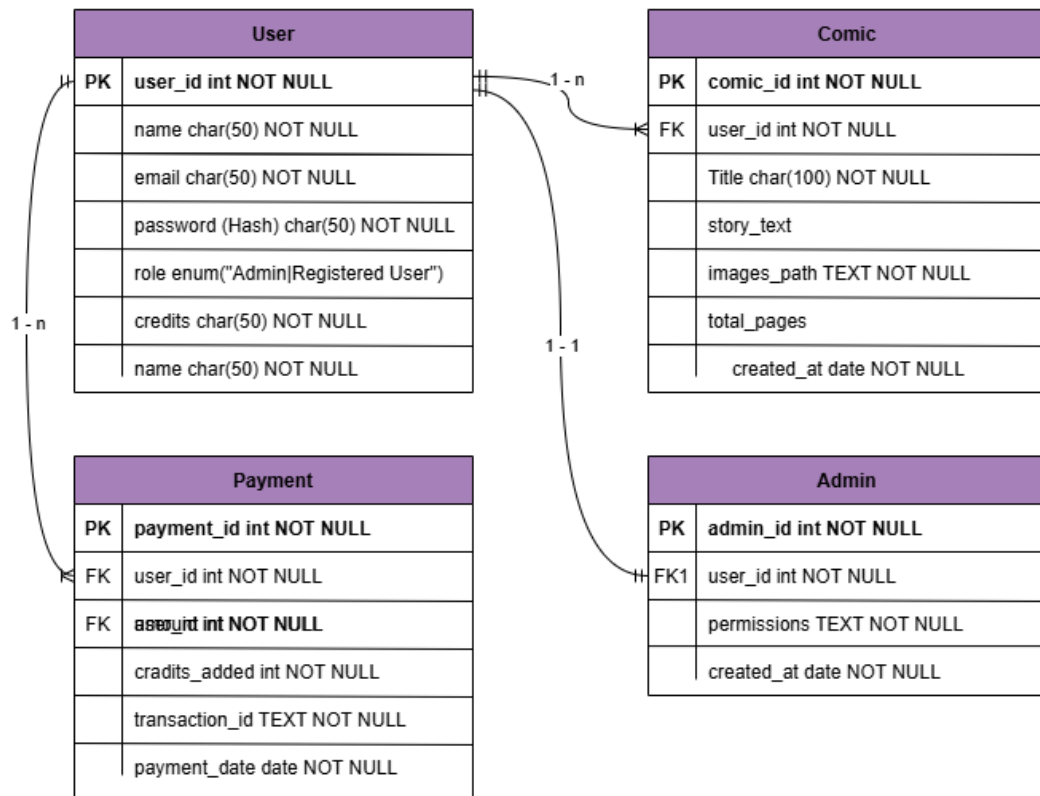


Figure 3.6: Entity Relationship Diagram (ERD)

Data Flow Diagram (DFD)

Here's how I structured the Data Flow Diagram (DFD) for my database based on the system explanation above.

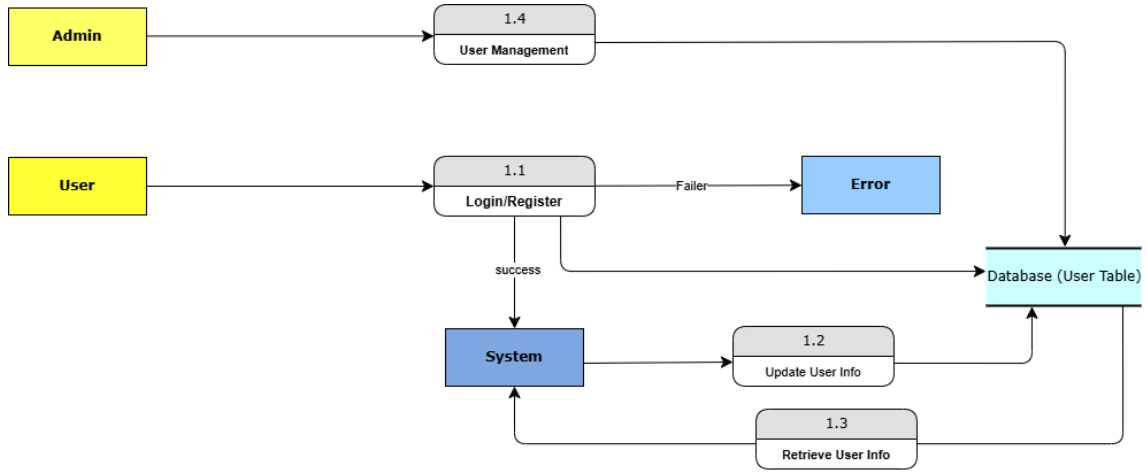


Figure 3.7: USER Database (DFD)

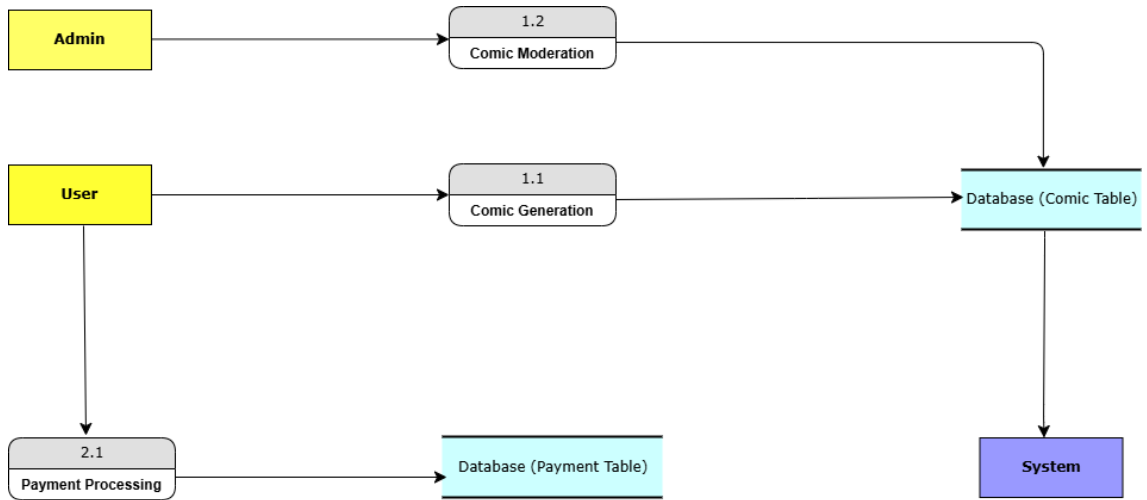


Figure 3.8: Comic & Payment Database (DFD)

3.1.6 System Interaction

User Sequence Diagram

The Sequence Diagram for this system illustrates the interaction flow between Guest Users, Registered Users, Admins, and the Backend System. It outlines key processes such as registration, login, comic generation, payment processing, and User management.

- Guest Users can enter text and generate a single image.
- Registered Users can log in, generate multiple images, manage their profile, view past comics, purchase credits, and download comics.

Each interaction involves API calls between the front-end and back-end, ensuring smooth authentication, comic generation, database updates, and payment processing.

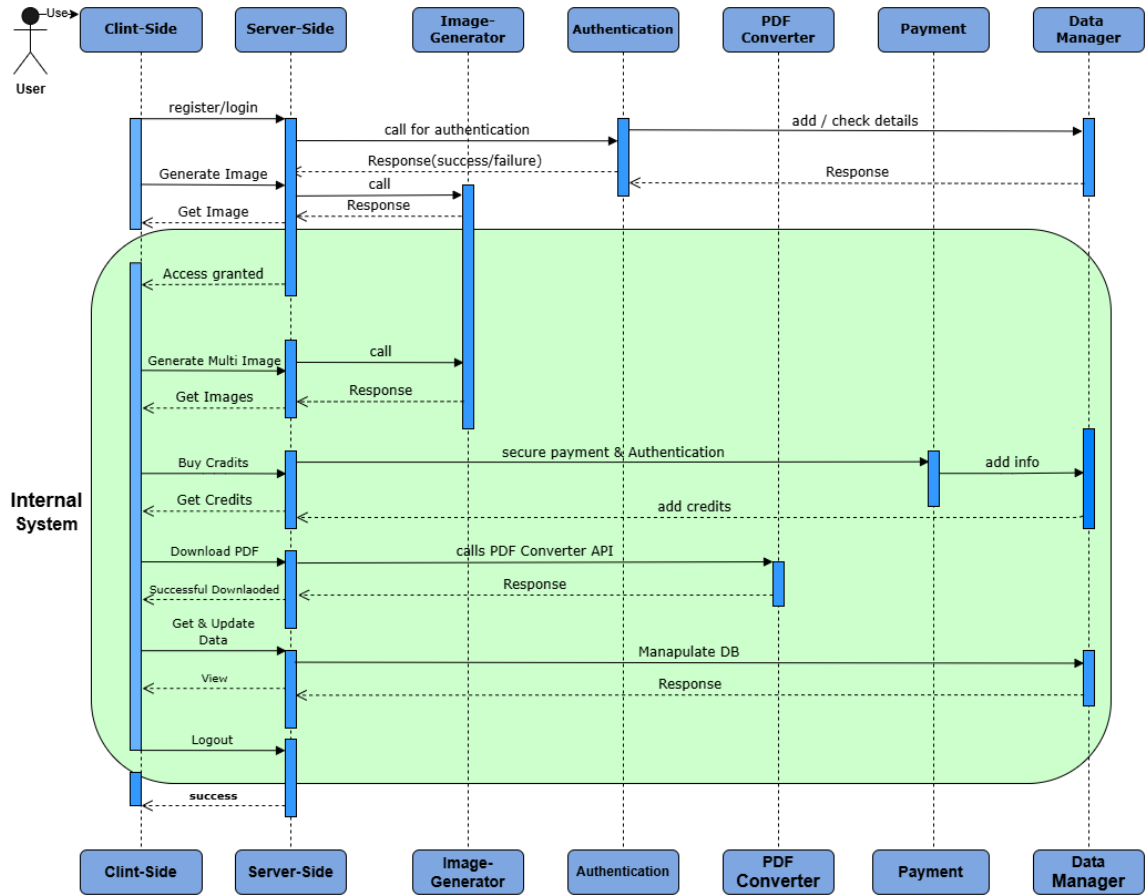


Figure 3.9: Sequence Class Diagram

Component Diagram

The Component Diagram of our system shows how different parts of the system work together. The User Interface (Frontend) connects to the Backend API, which handles user requests and sends data where needed. The AI Comic Generation Model creates images based on user input, while the Database stores user, comic, and payment details. The Payment Gateway processes transactions, and the Email Service sends generated comics to users. This diagram helps understand how each part of the system interacts to keep everything running smoothly.

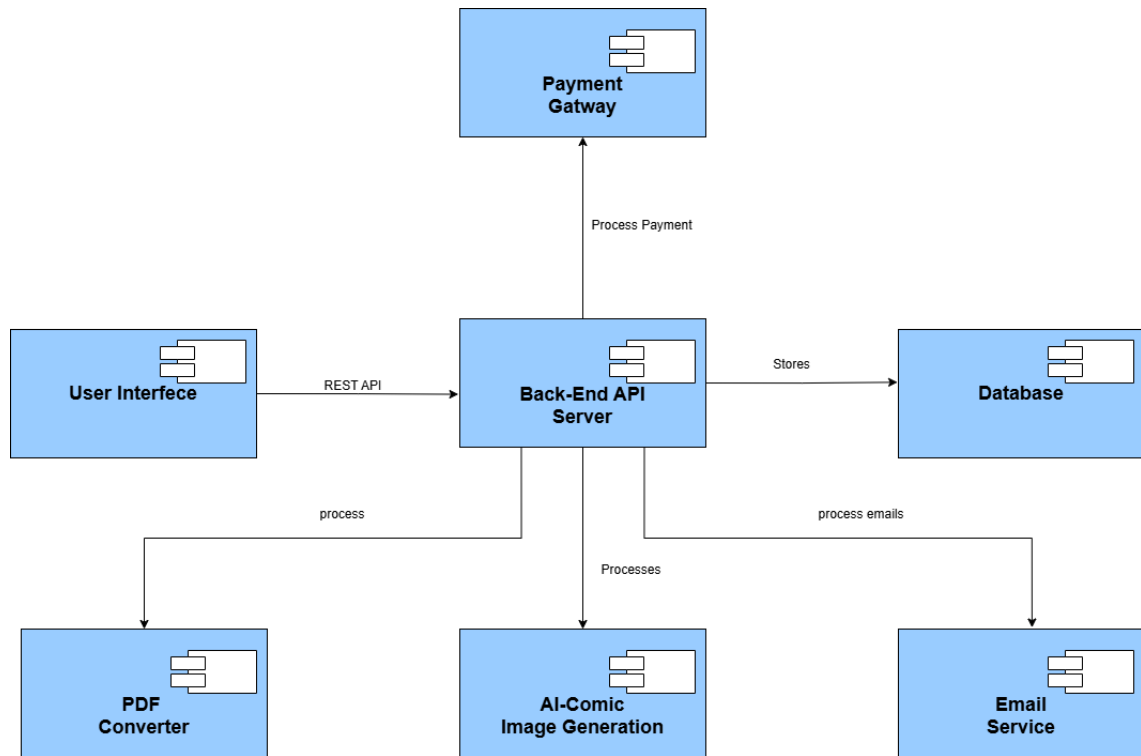


Figure 3.10: Component Diagram Of the System

3.2 Implementation

Chapter 4

Conclusion

Acknowledgements

In case your thesis received financial support from a project or the university, it is usually required to indicate the proper attribution in the thesis itself. Special thanks can also be expressed towards teachers, fellow students and colleagues who helped you in the process of creating your thesis.

Appendix A

Simulation results

List of Figures

3.1	System Architecture	6
3.2	Guest Users Front End Diagram	7
3.3	Registered Users Front End Diagram	8
3.4	Use Case Diagram	10
3.5	API Design Of the System	11
3.6	Entity Relationship Diagram (ERD)	12
3.7	USER Database (DFD)	13
3.8	Comic & Payment Database (DFD)	13
3.9	Sequence Class Diagram	14
3.10	Component Diagram Of the System	15

List of Tables

List of Algorithms

List of Codes